No, seriously, why is it important to us? You will have to wait a few paragraphs, then we will show you in practice.

### 7.7.3 Acquiring a Lock

So, now that we have an idea of what a lock is, we are ready to take a look at the modified version of the `set` method, in listing 7.10

As you can see, as soon as we enter the method, we lock our resource (the whole cache) for writing. What does it mean? When a write lock is set on a resource, the thread holding the lock is the only one that can write to *and* read from that resource. It means that all other threads, either trying to read or write on the resource, are going to have to wait till current lock holder releases it.

The second instruction here is a `try`, with a `finally` block at the end of the method (lines marked with 4 and 5), where we release the lock. This is necessary to prevent deadlock: if any of the operation inside the try fails, we still release the lock before exiting the function, so that other threads can acquire the lock and not be blocked anymore.

**Listing 7.10 Java concurrent LFU Cache set**

```
public boolean set(Key key, Value value) {

writeLock.lock();
              #1
  try
{
         #2
```